Today

Welcome to your next lesson: Methods and the role of self.

I'm preparing your personalized lesson based on what you've already learned, what's next, and the best way to help you practice.

Step by step, you’re moving forward.

Imagine your Pet object from earlier 🐶

So far, a pet can exist (as an object), but it can’t do anything yet.

To make it do something (like bark or greet), we put a method inside the class. A method is just a function defined inside a class that an object can call.

Very first step (no self yet, just the idea):

A method is an action that every object of that class knows how to do.

For example, in English:

Class (blueprint): Pet
Object: my_pet
Method idea: speak → “Woof!”
Before we touch code, I want to check your mental model:

If my_pet is an object, which of these feels more like “an action the pet can do”:

Pet
my_pet
speak

You can just answer with one of those three words.

speak

Exactly — speak is the action 🔓 Method idea unlocked.

So now let’s turn that action into real Python code inside a class.

Here’s the key idea, in simple words:

A method is a function written inside a class, and its first parameter is called self.

You can think of self as: “the object that is talking about itself.”

Soon we’ll write something like def speak(self): inside class Pet:, and then my_pet.speak() will trigger that behavior.

Quick check before I show any code:

When you later call the method like my_pet.speak(), do you think you will need to pass something for self, like my_pet.speak(self)? Or does Python handle self for you automatically?

You can answer in your own words, no need to be formal.

no its passed for you

Perfect — that’s exactly right 🎯

self is not something you pass; Python connects it automatically when you do my_pet.speak().

Now let me show you the full picture, step by step. Here’s the structure we’re aiming for (I’ll just describe it first, not ask you to code yet):

Class blueprint section (top of the file):

```
class Pet:
    def speak(self):
        """Make the pet speak."""
        print("Woof! 🐶")
```

Main program section (bottom of the file):

```
# main program
my_pet = Pet()
my_pet.speak()
```

Simple flow:

Pet class (blueprint) → create my_pet → call my_pet.speak() → it prints Woof! 🐶

Does this high-level flow make sense so far, before we practice in the editor?

Yes, it makes sense

Nice 👍 Let’s zoom in on just one new thing now: the method definition with self.

Inside the Pet class, we write:

Python

```
class Pet:
    def speak(self):
        """Make the pet speak."""
        print("Woof! 🐶")
```

One step at a time:

def speak(self): → defines a method named speak

```
self → means “the object that is calling this method”
The docstring """Make the pet speak.""" is just a description, not printed output
print("Woof! 🐶") is the actual behavior
Make sense that this is the blueprint for the action, not the action itself yet?


If yes, I’ll next show you how my_pet.speak() “activates” this method.

Yes, show me the method call

Great, let’s “wake up” the pet now 🐶


We’ll keep the two sections clearly separated:

Python
class Pet:
    def speak(self):
        """Make the pet speak."""
        print("Woof! 🐶")


# main program
my_pet = Pet()      # create an object from the Pet class
my_pet.speak()      # call its instance method
Step by step:


my_pet = Pet() → makes a Pet object.
my_pet.speak() → calls the speak method on that object.
Python secretly does: “use my_pet as self”, then runs the body and prints Woof! 🐶.

So my_pet.speak() is the activation of the behavior.

Before we jump into code practice, does this line-by-line flow feel clear, or is there one line you’d like me to re-explain more slowly?

The flow is clear

Awesome. Let’s lock in what each piece is before you code 🧩


```

Using this code:

Python
```
class Pet:
    def speak(self):
        """Make the pet speak."""
        print("Woof! 🐶")


# main program
my_pet = Pet()
my_pet.speak()
```
Here’s the mapping:

Pet → the class (blueprint)
my_pet → the object (one pet)
speak (inside the class) → the method (an action pets can do)
self → “the object that is talking about itself” when the method runs
my_pet.speak() → the method call that triggers the behavior (print("Woof! 🐶"))
Next step, I’d like you to try this yourself in the code editor:

Open the editor (</> button).
Type that exact code in Python.
Hit Run first to check it works.
Then hit Submit code so I can see it.

After you submit, I’ll give feedback and then we’ll try a tiny variation.

Python
```
class Pet:
    def speak(self):
        """Make the pet speak."""
        print("Woof! 🐶")


# main program
my_pet = Pet()
my_pet.speak()
```
Nice work, Ron — that’s a perfect first method ✅

You defined the class, included self correctly, added a

docstring, and called my_pet.speak() to trigger the behavior. Output looks exactly right: Woof! 🐶

Now, I want to focus just on self for a moment, with a simple idea:

When you write my_pet.speak(), Python secretly turns it into:

“Call speak, and use my_pet as self inside that method.”

You never type self in the call; you only write it in the method definition.

So in this call:

Python
```
my_pet.speak()
```
inside the method, self is basically my_pet.

Does that “self = the object that called the method” idea feel solid, or a bit fuzzy?

Answer with “solid” or “fuzzy” and I’ll adjust how deep we go.

Solid

Love it. Since your self idea is solid, let’s stretch your method skills a bit 🔧

We’ll keep the same structure, but change the behavior.

Right now you have:

Python
```
class Pet:
    def speak(self):
        """Make the pet speak."""
        print("Woof! 🐶")
```
I want you to:

Keep the class name Pet.

Keep the method name speak and self as the first parameter.

Change the method so it prints two lines, like:

First line: "The pet is about to speak..."
Second line: "Woof! 🐶"

So inside the method you'll have two print calls, one after the other.

Your turn 🎯

In the code editor, update your code to do this, Run it, then Submit code so I can check it.

Python

```
class Pet:
    def speak(self):
        """Make the pet speak."""
        print("The pet is about to speak...")
        print("Woof! 🐶")


# main program
my_pet = Pet()
my_pet.speak()
```

That's perfect, Ron 🎯

You kept self in the right place, kept the method name, and changed the behavior with two print calls. Output matches exactly.

Let's do one more small twist to be sure everything about calling methods is solid.

Look at these four lines (don't run them, just think):

```
Pet.speak
my_pet.speak()
my_pet = Pet
self.speak()
```

Only one of them actually activates the method on your pet object in the way you've been doing.

Which one do you think it is?

You can just answer with 1, 2, 3, or 4.

2

What do you think? Pick the line that actually makes the pet do its action 🐶👇

Your pet object is ready to act! Which line activates its instance method?

A.
Pet.speak

B.
my_pet.speak()

C.
my_pet = Pet

D.
self.speak()

Exactly, my_pet.speak() is the one that truly activates the method 🎉

Quick breakdown:

Pet.speak → just refers to the function on the class, doesn't call it.
my_pet.speak() → calls the method on the my_pet object, so Python passes my_pet in as self and runs the print lines.
my_pet = Pet → assigns the class itself, doesn't even create an object (missing ()).
self.speak() → only makes sense inside another method; it's not how you normally call the method from your main program.

You've hit all the goals for this lesson:

Define a class with a method that has self first ✅
Understand self as "the object talking about itself" ✅
Create an object and call its instance method with dot notation ✅

You've got this down. 💪

Are you ready to wrap up this lesson and move on to the next one, or do you want one more quick practice with a different class (like a Car method) before we finish?

I'm ready to move on

Nice work, Ron — lesson complete 🎯

You can now:

Define a class with a method that has self as the first parameter
Explain self as "the object talking about itself"
Create an object and call its method with dot notation (like my_pet.speak())

This is a big building block for everything else in OOP, so you've set yourself up well.

Whenever you're ready, you can jump into the next lesson from the Learn tab. If anything about methods or self feels shaky later, just tell me what you want to revisit and we'll reinforce it step by step.

```
class Pet:
    def speak(self):
        """Make the pet speak."""
        print("The pet is about to speak...")
        print("Woof! 🐶")
```

```
366   # main program
367   my_pet = Pet()
368   my_pet.speak()
```